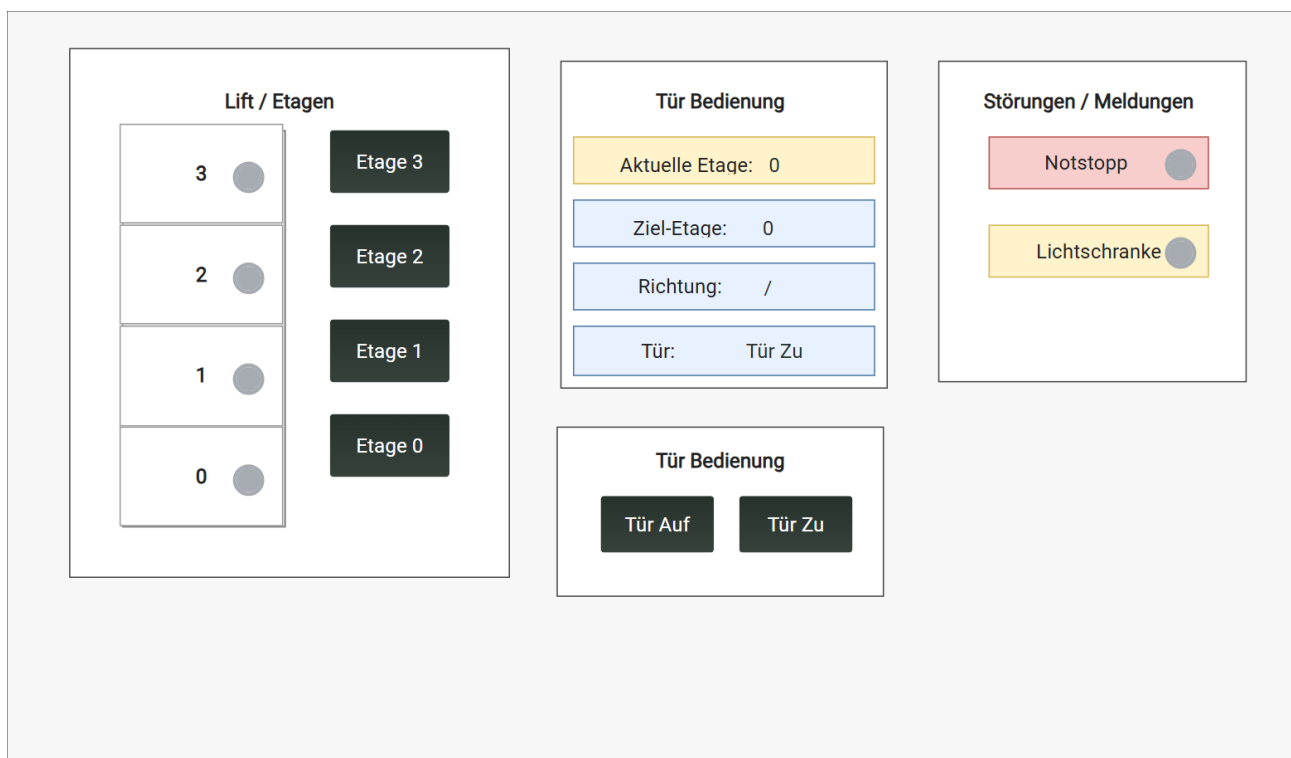


Case Study Steuerungstechnik

Aufzugssteuerung



Al Hassan Al Sebehawi und Niklaus Janett
Dipl. Systemtechniker/-in HF 13.0001B-2024
Abgabe 24.06.2026

Management Summary

In dieser Case Study wurde eine Aufzugsteuerung mit vier Etagen geplant, umgesetzt und getestet. Ziel des Projekts war es, eine praxisnahe Steuerungsaufgabe mit SPS, HMI, digitalen Eingängen und zwei Antrieben zu realisieren. Der Aufzug sollte Etagenanforderungen erkennen, die gewählte Zieletage anfahren und die Türsteuerung sicher in den Ablauf einbinden.

Für die Umsetzung wurde zuerst ein technisches Konzept mit Flussdiagramm, Zustandsautomat, Architekturübersicht und HMI-Skizze erstellt. Auf dieser Grundlage entstand im LASAL Machine Manager ein strukturiertes SPS-Projekt mit getrennten Bereichen für Hauptablauf, Aufzugmodul, Sensorik, Türantrieb, Aufzugantrieb und Visualisierung. Die Bedienung und Zustandsanzeige wurden über das HMI umgesetzt, sodass aktuelle Etage, Zieletage, Fahrtrichtung, Türzustand sowie weitere Signale nachvollziehbar dargestellt werden konnten.

Während der Realisierung wurden die Motoranbindung, die HMI-Kommunikation, die Git-Zusammenarbeit und mehrere Test- und Bugfix-Runden schrittweise aufgebaut.

Herausforderungen wie fehlende Bibliotheksdateien, Branch-Abstimmungen und die Anbindung einzelner Statuswerte konnten im Team gelöst werden. Insgesamt zeigt die Arbeit, dass eine saubere Planung, klare Schnittstellen, Versionsverwaltung und regelmässiges Testen entscheidend sind, um aus einer ersten Projektidee eine funktionierende und erweiterbare Aufzugsteuerung zu entwickeln.

Inhalt

1.	Einleitung	1
2.	Anforderungen und Ziele	1
2.1.	Muss-Ziele.....	2
2.2.	Kann-Ziele / Bonus-Ziele.....	2
3.	Planen und Konzipieren	3
3.1.	Flussdiagramm.....	3
3.2.	Zustandsautomat	5
3.3.	Architektur der Steuerung	6
3.4.	HMI-Konzept	7
3.5.	Geplante Programmstruktur.....	8
4.	Umsetzung.....	8
4.1.	Aufbau des SPS-Projekt	8
4.2.	Versionsverwaltung und Git-Test	9
4.3.	Aufbau der Architektur im SPS-Projekt.....	9
4.4.	Motoranbindung und Motion-Grundlage	10
4.5.	Aufbau der Sensorik und Hilfsklassen	11
4.6.	Umsetzung der Hauptsteuerung	11
4.7.	Schrittkette der Aufzugsteuerung.....	11
4.8.	Ansteuerung von Aufzug- und Türantrieb	12
4.9.	Bonus: Lichtschranke und Notstoppfunktion	12
4.10.	HMI-Anbindung und Visualisierung.....	13
4.11.	Testen, Bugfixes und Zusammenführung der Branches.....	14
5.	Schlussteil.....	15
5.1.	Niklaus	15
5.2.	5.1. Niklaus	Fehler! Textmarke nicht definiert.
5.3.	Hassan	16
	Aufwandsübersicht.....	17

Selbständigkeitserklärung.....	20
Abbildungsverzeichnis	21
Quellenverzeichnis.....	22
Einsatz von KI-Tools.....	23
Verwendete KI-Prompts.....	23

1. Einleitung

Im Rahmen der Case Study 4 im Modul Steuerungstechnik 2 entwickeln wir eine Aufzugsteuerung. Zu Beginn haben wir verschiedene mögliche Aufgabenstellungen diskutiert und überlegt, welches Projekt für uns technisch sinnvoll und interessant ist. Dabei war uns wichtig, dass die Aufgabe einen praktischen Bezug hat und wir die vorhandenen Komponenten der Steuerung sinnvoll einsetzen können.

Wir haben uns gezielt für eine Aufzugsteuerung entschieden, da wir in diesem Projekt mit Motoren, digitalen Eingängen, Ausgängen und einer logischen Ablaufsteuerung arbeiten können. Ein Aufzug eignet sich sehr gut, um typische Elemente der Steuerungstechnik umzusetzen, wie zum Beispiel Zustände, Verriegelungen, Fahrbefehle, Endlagen und Sicherheitsbedingungen. Dadurch können wir unser Wissen aus dem Unterricht praktisch anwenden und vertiefen.

Ziel dieser Case Study ist es, eine funktionierende Steuerung für einen Aufzug zu entwickeln, zu programmieren und zu testen. Dabei soll der Aufzug je nach Eingabe korrekt reagieren, die gewünschte Position anfahren und die Abläufe nachvollziehbar darstellen. Zusätzlich soll die Umsetzung so aufgebaut sein, dass sie verständlich dokumentiert und bei Bedarf erweitert werden kann.

2. Anforderungen und Ziele

In dieser Case Study wird eine Aufzugsteuerung mit vier Etagen simuliert. Die Anlage soll typische Funktionen eines einfachen Aufzugs abbilden und dabei mit zwei physischen Antrieben arbeiten. Ein Antrieb wird für die Bewegung des Aufzugs zwischen den Etagen verwendet. Der zweite Antrieb dient als Türantrieb zum Öffnen und Schliessen der Aufzugtür.

Die Bedienung der Aufzuganlage erfolgt über digitale Eingänge sowie über das HMI. Die vier Wahltaster beziehungsweise Schalter werden verwendet, um den Aufzug von den einzelnen Etagen aus zu bestellen. Dadurch wird eine Aussenbedienung im Treppenhaus simuliert. Wird eine Etage bestellt, soll der Aufzug diese Anforderung erkennen und die entsprechende Etage anfahren.

Zusätzlich werden digitale Taster für die Funktionen „Tür auf“ und „Tür zu“ verwendet. Damit kann die Tür manuell geöffnet oder geschlossen werden. Diese Bedienung soll nur dann möglich sein, wenn sich der Aufzug in einem sicheren Zustand befindet, zum Beispiel im Stillstand an einer Etage.

Das HMI dient zur übersichtlichen Darstellung und zusätzlichen Bedienung der Anlage. Auf dem HMI sollen vier Taster für die Auswahl der Zieletage vorhanden sein. Damit kann im HMI bestätigt beziehungsweise ausgewählt werden, in welche Etage der Aufzug fahren soll. Zusätzlich sollen auf dem HMI ebenfalls die Funktionen „Tür auf“ und „Tür zu“ vorhanden sein. Wichtige Zustände wie aktuelle Etage, Zieletage, Fahrtrichtung und Türzustand sollen auf dem HMI ersichtlich dargestellt werden.

2.1. Muss-Ziele

Die Aufzugsteuerung muss vier Etagen verwalten können. Jede Etage soll über einen eigenen Wahltaster beziehungsweise Schalter bestellt werden können. Diese Schalter simulieren die Aussenbedienung des Aufzugs im Treppenhaus.

Der Aufzug muss eine erkannte Etagenforderung verarbeiten und zur entsprechenden Etage fahren können. Der Aufzugsantrieb muss abhängig von der aktuellen Position und der gewählten Zieletage in die richtige Richtung angesteuert werden. Befindet sich die Zieletage oberhalb der aktuellen Etage, soll der Aufzug nach oben fahren. Befindet sich die Zieletage unterhalb der aktuellen Etage, soll der Aufzug nach unten fahren. Sobald die gewünschte Etage erreicht ist, muss der Aufzug stoppen.

Der Türantrieb muss in die Steuerung integriert werden. Die Tür soll über digitale Taster sowie über das HMI geöffnet und geschlossen werden können. Während der Fahrt darf die Tür nicht geöffnet werden. Der Aufzug darf nur fahren, wenn die Tür geschlossen ist.

Das HMI muss die Aufzuganlage verständlich darstellen. Dazu gehören mindestens die Anzeige der aktuellen Etage, die Anzeige der gewählten Zieletage, der Fahrzustand des Aufzugs und der Zustand der Tür. Zusätzlich sollen auf dem HMI vier Etagentaster sowie die Taster „Tür auf“ und „Tür zu“ vorhanden sein.

Das SPS-Programm soll klar strukturiert sein. Die wichtigsten Zustände der Anlage sollen eindeutig abgebildet werden. Dazu gehören Stillstand, Fahrt nach oben, Fahrt nach unten, Tür öffnen, Tür offen und Tür schliessen.

2.2. Kann-Ziele / Bonus-Ziele

Als Bonus soll eine Notstopp-Funktion umgesetzt werden. Diese kann über einen Notstopptaster oder über eine simulierte Lichtschranke ausgelöst werden. Wird der Notstopp betätigt, muss die Anlage sofort stoppen und in einen sicheren Zustand wechseln.

Zusätzlich kann eine Sicherheitsfunktion über eine Lichtschranke umgesetzt werden. Wird die Lichtschranke während des Schliessens der Tür unterbrochen, soll die Tür nicht weiter schliessen. Stattdessen soll die Tür wieder öffnen oder im geöffneten Zustand bleiben.

Ein weiteres Bonus-Ziel ist eine erweiterte Darstellung am HMI. Dabei sollen die Etagen, die aktuelle Position des Aufzugs, die gewählte Zieletage, die Türbewegung und mögliche Störungen möglichst übersichtlich visualisiert werden.

Falls genügend Zeit vorhanden ist, kann die Steuerung erweitert werden, sodass mehrere Etagenanforderungen nacheinander verarbeitet werden. Dadurch würde die Simulation näher an eine reale Aufzugsteuerung herankommen.

3. Planen und Konzipieren

Für die Umsetzung der Aufzugssteuerung wurde zuerst ein Konzept erstellt. Ziel der Planungsphase war es, die Hauptfunktion der Anlage verständlich zu strukturieren und eine klare Grundlage für die spätere SPS-Programmierung zu schaffen. Dabei wurden die Anforderungen aus der Aufgabenstellung in einzelne Teilfunktionen aufgeteilt.

Im Zentrum der Planung standen drei Punkte: der grundsätzliche Ablauf der Steuerung, die Zustände der Anlage und die Aufteilung der einzelnen Systemkomponenten. Dadurch konnte vor der Programmierung festgelegt werden, welche Signale benötigt werden, welche Zustände die Anlage besitzen muss und wie HMI, Steuerung, Antriebe und Sensorik zusammenarbeiten.

3.1. Flussdiagramm

Als erster Planungsschritt wurde ein Flussdiagramm erstellt. Dieses zeigt den grundsätzlichen Ablauf der Aufzugssteuerung. Dabei wird dargestellt, wie eine Etagenwahl erkannt wird, wie die Ziel-Etage gespeichert wird und unter welchen Bedingungen der Aufzug fahren darf.

Wichtig ist dabei die Türverriegelung. Der Aufzug darf nur fahren, wenn die Tür geschlossen ist. Ist die Tür offen, wird zuerst der Türantrieb zum Schliessen angesteuert. Erst danach wird der Aufzugsmotor aktiviert. Sobald die gewünschte Ziel-Etage erreicht ist, stoppt der Aufzug und die Tür wird geöffnet.

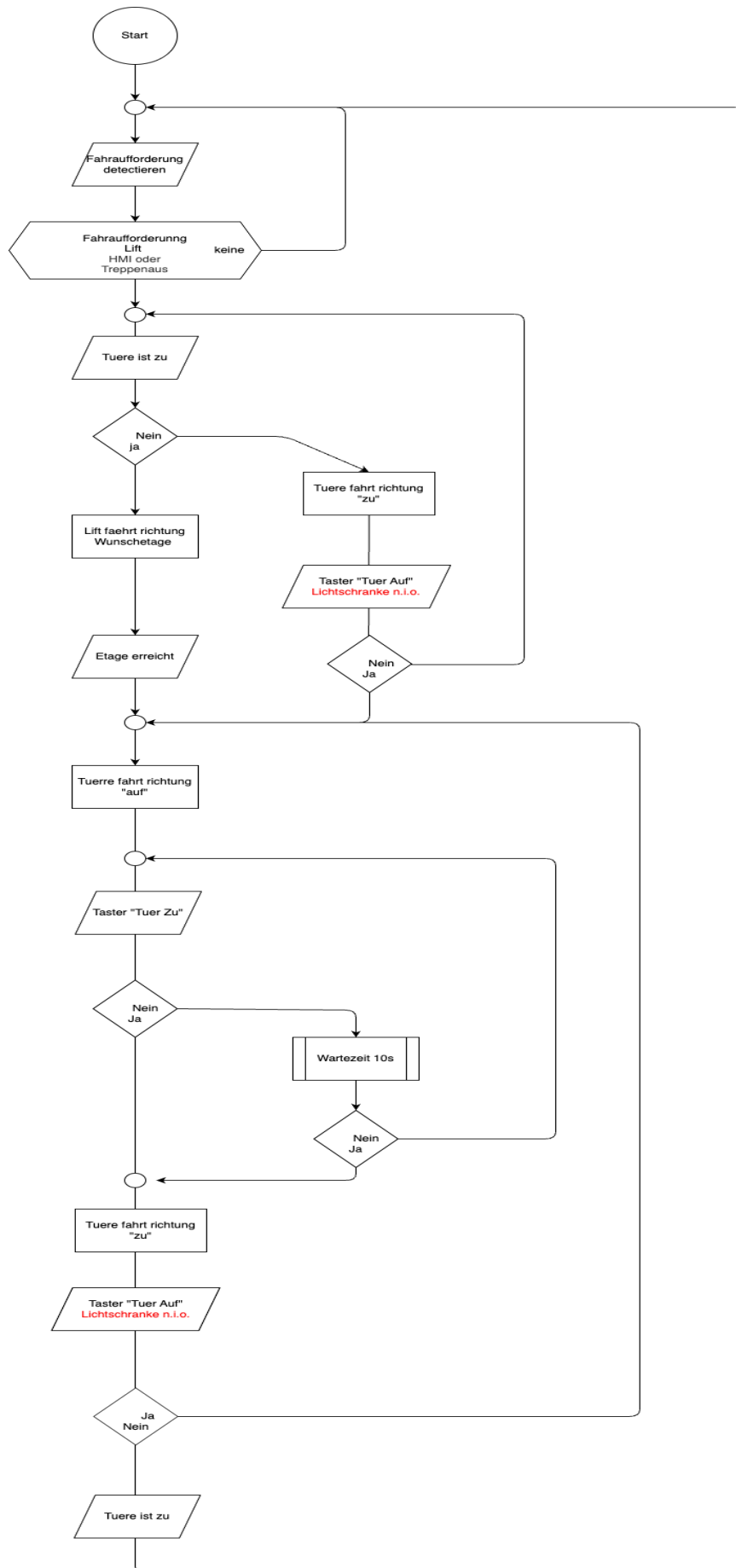


Abbildung 1: Flussdiagramm der Aufzugssteuerung

Wird die Ziel-Etage erreicht, wird der Aufzugmotor ausgeschaltet und die Tür geöffnet. Nach einer Wartezeit wird die Tür wieder geschlossen. Die Bonusfunktion Notstopp beziehungsweise Lichtschranke wurde im Zustandsautomaten separat gekennzeichnet, da sie nicht zur Hauptfunktion gehört, sondern als Erweiterung vorgesehen ist.

3.3. Architektur der Steuerung

Neben dem Ablauf wurde auch die Architektur der Steuerung geplant. Die Architekturzeichnung zeigt, welche Hauptkomponenten im System vorhanden sind und wie sie miteinander verbunden sind. Dabei wird zwischen Hauptablauf, HMI, Aufzug-Modul, Türantrieb, Aufzugantrieb sowie Etagen- und Türsensorik unterschieden.

Der Hauptablauf steuert das Aufzug Modul. Das Aufzug Modul koordiniert den Türantrieb, den Aufzugantrieb und die Rückmeldungen der Etagen- und Türsensorik. Das HMI kommuniziert mit dem Hauptablauf.

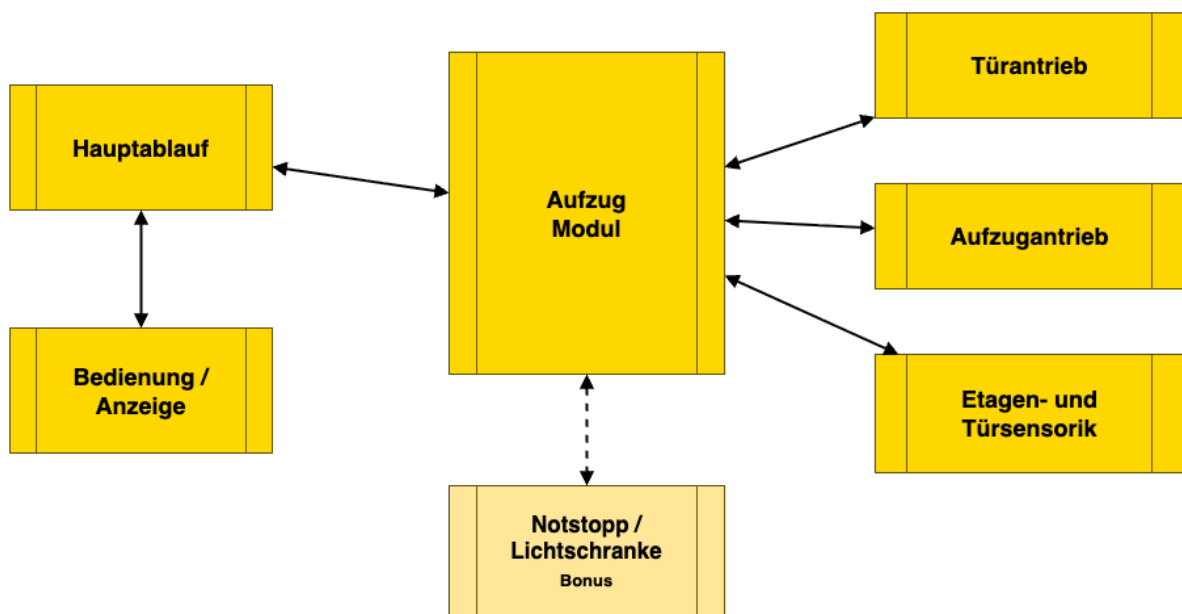


Abbildung 3: Architektur der Aufzugssteuerung

Der Hauptablauf steuert das Aufzug-Modul. Das Aufzug-Modul koordiniert den Türantrieb, den Aufzugantrieb und die Rückmeldungen der Sensorik. Die Bedienung und Anzeige erfolgt über das HMI. Über das HMI sollen Zustände wie aktuelle Etage, Ziel-Etage, Fahrtrichtung und Türzustand sichtbar sein.

Die Architektur hilft dabei, die spätere Programmstruktur sauber aufzubauen. Die Steuerungslogik, die HMI-Anzeige und die Hardware-Signale sollen möglichst klar getrennt werden. Dadurch wird das Programm übersichtlicher und kann einfacher getestet oder erweitert werden.

3.4. HMI-Konzept

Für die Visualisierung wurde geplant, die wichtigsten Zustände der Anlage auf dem HMI darzustellen. Das HMI soll nicht nur zur Anzeige dienen, sondern auch einfache Bedienfunktionen ermöglichen.

Geplant sind folgende Anzeigen und Bedienelemente:

- Anzeige der aktuellen Etage
- Anzeige der gewählten Ziel-Etage
- Anzeige der Fahrtrichtung
- Anzeige des Türzustands
- Anzeige von Störungen oder Notstopp als Bonusfunktion
- Taster für die Etagenwahl
- Taster für Tür öffnen und Tür schliessen

HMI Skizze Aufzugssteuerung

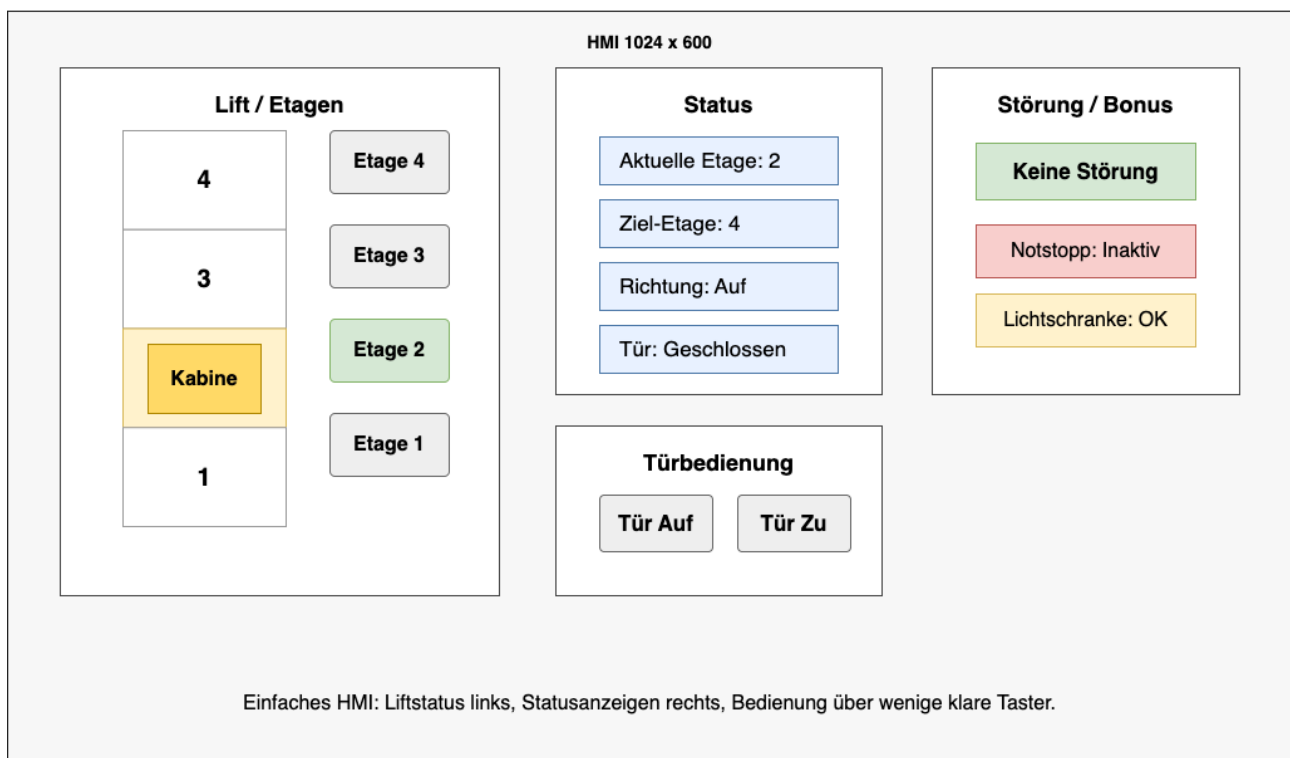


Abbildung 4: HMI-Skizze

Das HMI soll so aufgebaut sein, dass der Zustand der Anlage schnell erkennbar ist. Dadurch kann während der Inbetriebnahme und beim Testen kontrolliert werden, ob die SPS-Logik korrekt arbeitet.

3.5. Geplante Programmstruktur

Für die SPS-Programmierung ist eine klare Struktur vorgesehen. Die zentrale Steuerung soll als Zustandsautomat umgesetzt werden. Die Zustände werden mit einem ENUM abgebildet und in einer CASE-Struktur verarbeitet.

Die wichtigsten Programmteile sind:

- Erfassung der Eingänge
- Flankenerkennung der Etagenwahlschalter
- Speichern der Ziel-Etage
- Bestimmen der Fahrtrichtung
- Ansteuerung des Aufzugantriebs
- Ansteuerung des Türantriebs
- Verarbeitung der Tür- und Etagenrückmeldungen
- Bereitstellung der HMI-Variablen
- Verarbeitung der Bonusfunktion Notstopp, falls umgesetzt

Durch diese Struktur kann die Hauptfunktion zuerst umgesetzt und getestet werden. Erst wenn diese zuverlässig funktioniert, werden die Bonusfunktionen ergänzt. Dieses Vorgehen entspricht dem iterativen Ansatz der Case Study: planen, umsetzen, testen, verbessern und dokumentieren.

4. Umsetzung

4.1. Aufbau des SPS-Projekt

Nachdem die Analyse und die Planung abgeschlossen waren, wurde als erster Umsetzungsschritt das SPS-Projekt aufgebaut. Dafür wurde im LASAL Machine Manager eine neue Solution erstellt, in der die CP112-Steuerung und das HMI gemeinsam verwaltet werden. Damit war die Grundstruktur vorhanden, um später die SPS-Logik, die Antriebe und die Visualisierung im gleichen Projekt weiterzuentwickeln.

Anschliessend wurde das Projekt in das gemeinsame GitHub-Repository hochgeladen, auf das beide Projektmitglieder Zugriff hatten. Die Repository-Struktur wurde in die Ordner Doc und src aufgeteilt. Im Ordner Doc wurden die Dokumentation, Aufgabenstellung und Analyse abgelegt. Im Ordner src befanden sich die LASAL-Projektdateien für SPS und HMI. Zusätzlich wurden eine .gitignore- und eine .gitattributes-Datei ergänzt, damit die LASAL-Dateien sauber versioniert werden konnten.

Doc	Aufwandsübersicht	18 hours ago
src	Merge branch 'main' of https://github.com/niklaus619/CS_...	last week
.gitattributes	gitignore und gitattributes hinzugefügt für LASAL	3 weeks ago
.gitignore	gitignore	2 weeks ago

Abbildung 5: Ordnerstruktur des GitHub-Repositorys

4.2. Versionsverwaltung und Git-Test

Nachdem das Repository erstellt war, wurde es auf den verschiedenen Arbeitsrechnern sowie auf einer Virtual Machine geklont. Damit wurde überprüft, ob alle Projektdateien korrekt heruntergeladen werden und ob die Zusammenarbeit über Git funktioniert. Der Test war erfolgreich, sodass beide Projektmitglieder mit dem gleichen Projektstand weiterarbeiten konnten.

4.3. Aufbau der Architektur im SPS-Projekt

nach dem erfolgreichen Git-Test wurde die in der Analyse geplante Softwarearchitektur in LASAL übernommen. Dafür wurden die grundlegenden Klassen des SPS-Projekts erstellt und im Hauptnetzwerk miteinander verbunden. Dazu gehörten unter anderem die HMI-Klasse, der Hauptablauf, das Aufzugmodul, die Etagen- und Türsensorik sowie die Antriebe.

Im Hauptnetzwerk ist ersichtlich, wie die einzelnen Bausteine miteinander kommunizieren. Die Eingänge der Etagenwahlschalter werden an die HMI-Klasse weitergegeben. Von dort gelangen die Anforderungen zum Hauptablauf und anschliessend zum Aufzugmodul. Das Aufzugmodul koordiniert die Sensorik, die Türsteuerung und den Aufzugantrieb.

Dieser Schritt wurde gemeinsam durchgeführt, damit die Programmstruktur von Anfang an für beide Projektmitglieder verständlich war. Dadurch entstand eine klare Grundlage für die spätere Umsetzung der einzelnen Funktionen.

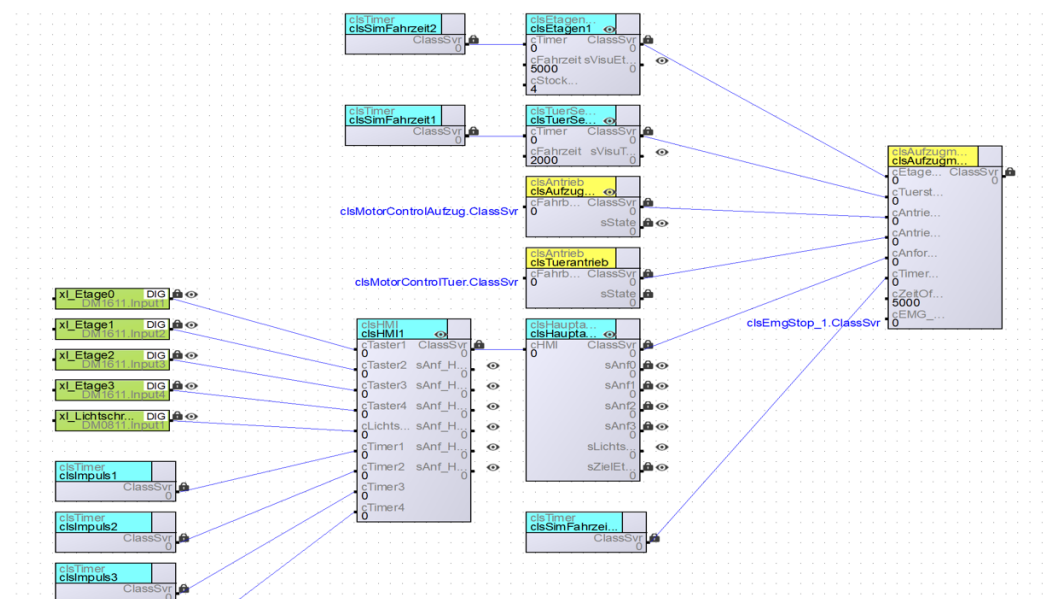


Abbildung 6: Hauptnetzwerk der SPS-Architektur in LASAL

4.4. Motoranbindung und Motion-Grundlage

Für die Motoranbindung wurden die Unterlagen des Dozenten als Grundlage verwendet. Die vorhandenen Beispiele und Vorgaben zur Motion-Ansteuerung wurden auf unser SPS-Projekt übertragen und an unsere Aufzugsteuerung angepasst. Dabei wurden die benötigten Motion-Objekte, Achsen und Motorbausteine in das LASAL-Projekt eingebunden.

Die Motoranbindung wurde zuerst als Grundlage umgesetzt, damit später der Aufzugantrieb und der Türantrieb getrennt angesteuert werden konnten. Dabei war wichtig, dass beide Bewegungsrichtungen vorbereitet werden: eine positive und eine negative Fahrtrichtung.

Während der Arbeit traten Probleme mit Git auf. Einige Treiber- und Motion-Dateien wurden durch die .gitignore-Datei nicht richtig mit versioniert. Dadurch fehlten nach dem Pull auf einem anderen Rechner wichtige Dateien für die Motoranbindung. Das Problem wurde gelöst, indem die betroffenen Einträge in der .gitignore-Datei angepasst wurden. Anschliessend wurde eine vorherige funktionierende Version wiederhergestellt und die benötigten Dateien wurden korrekt ins Repository übernommen.

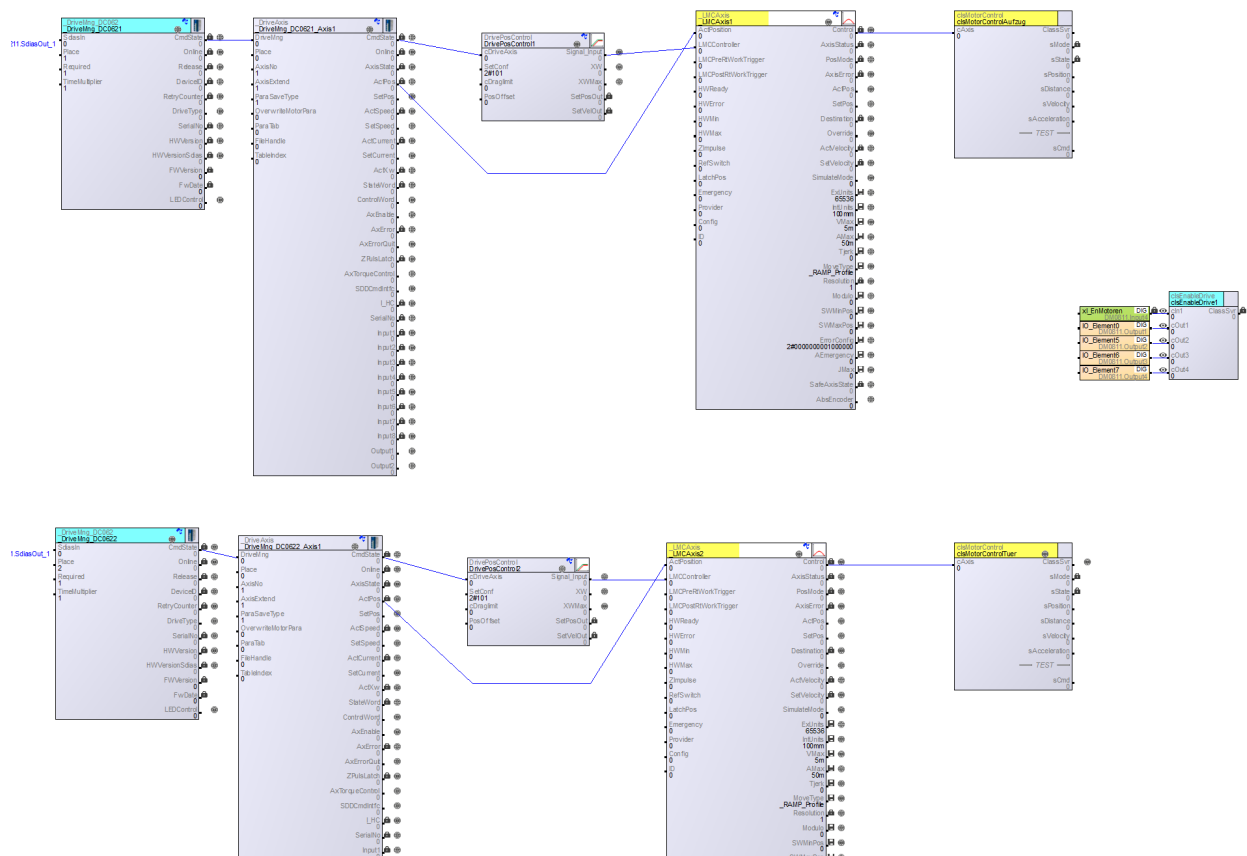


Abbildung 7: Eingebundene Motion-Objekte für die Motoranbindung

4.5. Aufbau der Sensorik und Hilfsklassen

Für die Simulation der Rückmeldungen wurden eigene Sensorikklassen erstellt. Dabei wurde zwischen der Türsensorik und der Etagensensorik unterschieden, damit die Sensorik sauber von der eigentlichen Ablaufsteuerung getrennt bleibt.

Für die Türsensorik wurde die Klasse `clsTuersensorik` erstellt. Diese simuliert die Endlagen „Tür offen“ und „Tür geschlossen“. Da keine echten Endlagensensoren verwendet wurden, wird die Verfahrzeit mit einem `clsTimer` nachgebildet. Beim Methodenaufwurf wird jeweils eine boolesche Variable für die Bewegung „auf“ oder „zu“ übergeben.

Für die Etagenrückmeldung wurde die Klasse `clsEtagensensorik` umgesetzt. Auch hier wird ein `clsTimer` verwendet, um das Erreichen einer Etage nach einer definierten Fahrzeit zu simulieren. Die Verfahrzeit ist als Client parametrierbar und kann dadurch ohne Änderung der Programmlogik angepasst werden.

Damit erhält die Hauptsteuerung klare Rückmeldungen wie „Tür offen“, „Tür geschlossen“ oder „Etage erreicht“.

4.6. Umsetzung der Hauptsteuerung

Der `clsHauptablauf` wurde als zentraler Datenknotenpunkt zwischen HMI und Aufzugmodul umgesetzt. In dieser Klasse werden die Etagenanforderungen gespeichert, abgefragt und bei Bedarf gelöscht.

Die Anforderungen werden über Methoden abgefragt. Dabei wird jeweils ein boolescher Parameter übergeben. Ist dieser Parameter `FALSE`, wird die Anforderung nur gelesen. Ist er `TRUE`, wird die entsprechende Anforderung nach der Abfrage zurückgesetzt.

Das Setzen der Anforderungen erfolgt über die Klasse `clsHMI`. Dort werden die Taster im zyklischen Task über eine Flankenerkennung ausgewertet und anschliessend an den Hauptablauf weitergegeben.

4.7. Schrittkette der Aufzugsteuerung

Die eigentliche Ablaufsteuerung des Aufzugs wurde als Schrittkette umgesetzt. Dabei wird der aktuelle Zustand über eine Schrittvariable verwaltet und in einer CASE-Struktur verarbeitet.

Dadurch bleibt klar ersichtlich, in welchem Zustand sich die Anlage gerade befindet.

Die Schrittkette prüft zuerst, ob eine Etagenanforderung vorhanden ist. Danach wird die aktuelle Etage mit der Zieletage verglichen und die Fahrtrichtung bestimmt. Bevor der Aufzug fährt, wird geprüft, ob die Tür geschlossen ist. Erst danach wird der Aufzugantrieb gestartet.

Sobald die gewünschte Etage erreicht ist, wird der Aufzug gestoppt und die Tür geöffnet. Nach der Offenhaltezeit wird die Tür wieder geschlossen. Erst nach erfolgreichem Schliessen der Tür wird die erledigte Etagenanforderung im Hauptablauf gelöscht.

Zusätzlich wurde in der Schrittkette bereits die Funktion „Tür auf“ integriert. Diese kann auch als Reaktion auf die Lichtschranke verwendet werden. Wird die Lichtschranke ausgelöst, kann damit das Schliessen der Tür unterbrochen und die Tür wieder geöffnet werden.

Ein bekanntes Problem besteht bei der simulierten Türzeit. Da die Türbewegung über eine feste Simulationszeit abgebildet wird, wäre ein dynamisches Verkürzen oder direktes Abbrechen der laufenden Türbewegung aufwendig umzusetzen. Deshalb läuft die simulierte Türbewegung auch nach einem frühzeitigen Tür-auf-Befehl noch kurz nach, bevor der neue Zustand vollständig übernommen wird.

4.8. Ansteuerung von Aufzug- und Türantrieb

Für die Ansteuerung der Antriebe wurde zuerst geprüft, ob die Fahrbewegung mit der Funktion `MOVE_INFINITE` umgesetzt werden kann. In der weiteren Umsetzung zeigte sich jedoch, dass die Arbeit mit `MOVE_ABS` und `MOVE_REL` einfacher in die bestehende Schrittkette integriert werden konnte.

Während der Tests stellte sich heraus, dass `MOVE_REL` für diese Anwendung kritisch war. Bei vielen aufeinanderfolgenden Testfahrten konnte sich der Positionszähler verschieben beziehungsweise überlaufen. Dadurch blieb der Antrieb teilweise stehen oder reagierte nicht mehr zuverlässig. Aus diesem Grund wurde die Ansteuerung auf `MOVE_ABS` umgestellt. Für beide Fahrtrichtungen wurden feste absolute Zielpositionen verwendet, eine in positiver und eine in negativer Richtung.

Die Antriebslogik wurde in der Klasse `clsAntrieb` umgesetzt. Daraus wurden zwei Objekte gebildet: eines für den Türantrieb und eines für den Aufzugantrieb. Diese Klasse wurde netzwerkübergreifend mit der `clsMotionControl` verbunden. Dadurch konnten beide Antriebe mit der gleichen Grundlogik angesteuert werden, während die Verwendung im Aufzugmodul getrennt blieb.

Gleichzeitig wurde beim Fahrbefehl nicht nur der reale Antrieb angesteuert. Sobald der Befehl über einen Methodenaufruf ausgelöst wurde, wurde zusätzlich ein Methodenaufruf an die Simulationsklasse gesendet. Dadurch konnten die simulierten Rückmeldungen wie Türendlage oder Etage erreicht parallel zur Antriebsbewegung erzeugt werden.

4.9. Bonus: Lichtschranke und Notstoppfunktion

Die Bonusfunktionen Lichtschranke und Notstopp wurden nachträglich in die bestehende Steuerung integriert. Dabei entstand zunächst ein Fehler, weil Teile der Logik ursprünglich im `Cyclic Task` geschrieben wurden und anschliessend im `Realtime Task` aufgerufen werden mussten. Dadurch mussten die betroffenen Programmteile angepasst und sauber in den richtigen Task übernommen werden.

Die Lichtschranke wurde nicht als Safety-Funktion im `Realtime Task` umgesetzt, da dafür auch die Eingänge der Safety-Card hätten umverdrahtet werden müssen. Stattdessen wurde sie parallel zur Funktion „Tür auf“ eingebunden. Die Lichtschranke wirkt damit wie ein zusätzlicher Tür-auf-Befehl und ist logisch über eine ODER-Verknüpfung mit dem Tür-auf-Taster verbunden.

Für den Notstopp wurde das Stoppverhalten in clsAntrieb und clsMotionControl erweitert. Dafür wurde eine zusätzliche Stoppfunktion erstellt, bei der der Bremsparameter sehr hoch gesetzt wurde. Dadurch wird ein schneller Stopp der Bewegung erreicht, vergleichbar mit einem Sofortstopp mit Bremsung.

Nach der Anpassung der Zyklusaufrufe konnte die Funktion am Modell getestet werden. Wird der Notstopp ausgelöst, stoppt der Lift. Nach dem Entrasten des Notstopps fährt der Lift direkt weiter, da keine separate Quittierfunktion umgesetzt wurde.

4.10. HMI-Anbindung und Visualisierung

Die HMI-Anbindung wurde hauptsächlich im Branch feature/VISU umgesetzt. Als Grundlage diente das HMI-Mockup aus der Analyse. Die geplanten Bedienelemente und Anzeigen wurden in LASAL VISU Designer nachgebaut und schrittweise mit der SPS verbunden.

Die Visualisierung wurde laufend an den aktuellen Stand der Steuerung angepasst. Sobald neue Funktionen in der SPS vorhanden waren, wurde gemeinsam besprochen, welche Werte und Schnittstellen für das HMI benötigt werden. Danach wurden die entsprechenden Server und Variablen erstellt und in der Visualisierung verwendet. Dazu gehörten unter anderem die Etagenwahl, die aktuelle Etage, die Fahrtrichtung, der Türstatus sowie die Bedienung für Tür auf und Tür zu.

Da noch wenig Erfahrung mit dem HMI-Design vorhanden war, mussten viele Elemente zuerst ausprobiert und getestet werden. Besonders die Anbindung der Variablen, Textschemes und Statusanzeigen wurde schrittweise aufgebaut. Durch dieses Vorgehen konnte die Visualisierung nach und nach erweitert werden, bis die wichtigsten Zustände der Aufzugsteuerung am HMI sichtbar und bedienbar waren.

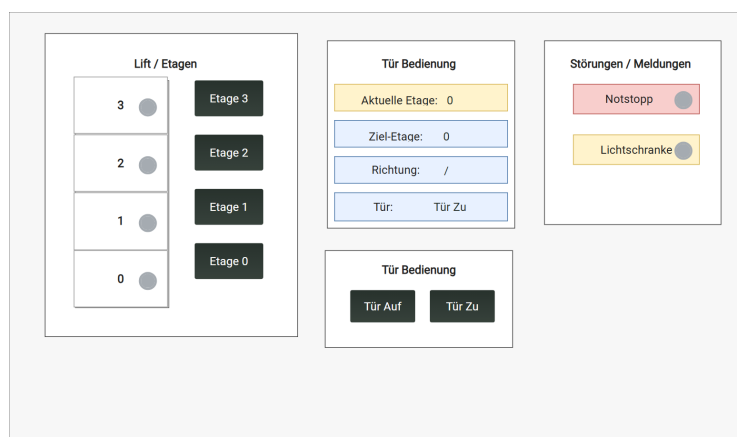


Abbildung 8: Ergebnis HMI

4.11. Testen, Bugfixes und Zusammenführung der Branches

Zu Beginn wurde der Grundstand auf dem main-Branch erstellt. Danach wurden für die parallele Arbeit eigene Branches abgeleitet. Auf dem Branch feature/Janett wurde hauptsächlich die Steuerungslogik entwickelt. Auf dem Branch feature/VISU wurde die HMI-Visualisierung aufgebaut und angebunden. Dadurch konnten beide Projektmitglieder gleichzeitig am Projekt arbeiten, ohne sich gegenseitig direkt zu überschreiben.

Damit die Visualisierung immer zum aktuellen Stand der Steuerung passte, wurden Änderungen aus feature/Janett regelmässig in feature/VISU übernommen. Dadurch konnten neue Funktionen der Steuerung direkt im HMI angebunden und getestet werden. Die Commit-Historie zeigt mehrere Merge-Schritte zwischen den beiden Branches, weil Steuerung und HMI laufend aufeinander abgestimmt werden mussten.

Nach dem Einbau neuer Funktionen wurde jeweils direkt getestet. Je nach Funktion erfolgte der Test entweder an der Steuerung, in der Simulation oder über das HMI. Dabei wurden zum Beispiel die Etagenwahl, die Fahrtrichtung, der Türstatus und die Reaktion auf Bedienbefehle überprüft. Gefundene Fehler wurden im Team besprochen und anschliessend direkt korrigiert. So konnten Bugfixes laufend in die Entwicklung einfließen, bevor die Branches wieder zusammengeführt wurden.

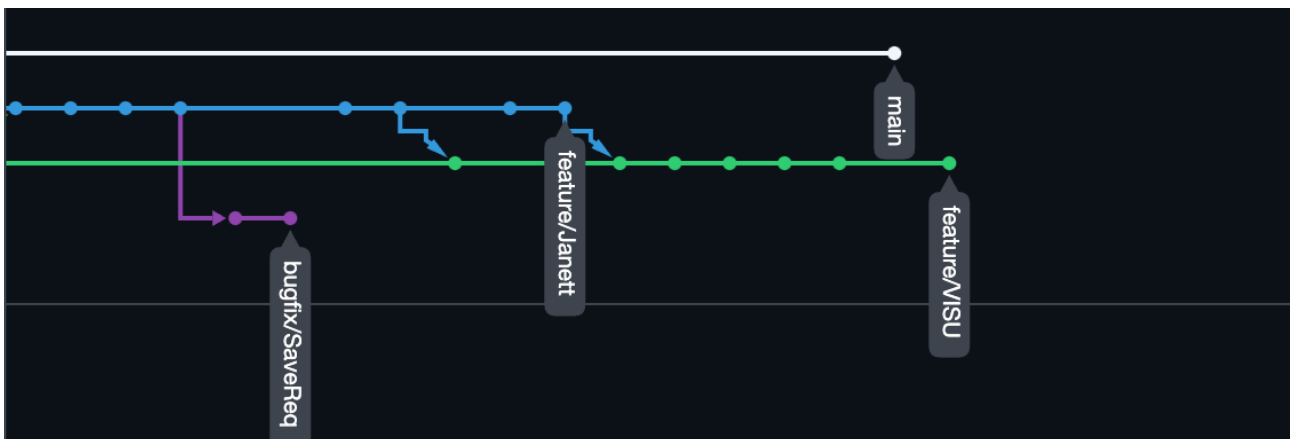


Abbildung 9: Git-Network

5. Schlussteil

Die Case Study war für uns sehr lehrreich, da wir eine komplette Steuerung von der Analyse bis zur Umsetzung gemeinsam entwickeln konnten. Besonders interessant war, dass wir das Wissen aus Steuerungstechnik 1 und 2 praktisch anwenden konnten. Dadurch konnten wir Schritt für Schritt nachvollziehen, wie aus einer geplanten Architektur eine funktionierende Steuerung entsteht.

Die Zusammenarbeit im Team hat gut funktioniert, da wir die Aufgaben sinnvoll aufgeteilt und uns regelmässig abgesprochen haben. So konnten wir parallel an der Steuerung, der HMI-Visualisierung und der Dokumentation arbeiten. Auch die Zusammenarbeit mit Git war für uns ein wichtiger Bestandteil des Projekts. Obwohl es zwischendurch Probleme mit Branches, Pulls, Pushes und fehlenden Dateien gab, konnten wir diese gemeinsam lösen und daraus viel lernen.

Technisch war vor allem die Anbindung der Motoren und die Arbeit mit Motion sehr spannend. Dabei konnten wir besser verstehen, wie Antriebe in ein SPS-Projekt eingebunden und später von der Steuerungslogik verwendet werden. Insgesamt hat uns die Case Study gezeigt, wie wichtig eine saubere Planung, klare Schnittstellen und regelmässiges Testen sind. Trotz einiger Herausforderungen hat das Projekt Spass gemacht und war für uns ein wertvoller Lernprozess.

5.1. Niklaus

Ich empfand die Case Study 4 als ein sehr technisches und interessantes Projekt. Besonders wertvoll war für mich, dass ich mein Verständnis für objektorientierte Programmierung in der SPS-Welt deutlich vertiefen konnte. Durch die Aufteilung in einzelne Klassen, Methoden und Objekte wurde für mich klarer, wie ein grösseres Steuerungsprogramm strukturiert aufgebaut werden kann. Vor allem das Zusammenspiel zwischen Hauptablauf, Aufzugmodul, Sensorik, Antriebsklassen und HMI hat mir geholfen, das Gesamtkonzept besser zu verstehen.

Die Arbeit mit der SIGMATEK-Umgebung war für mich teilweise herausfordernd. Da ich aus anderen Steuerungsumgebungen wie B&R, Beckhoff und TIA komme, musste ich mich zuerst an die Arbeitsweise in LASAL gewöhnen. Besonders die vielen einzelnen Einstellungen, Task-Zuordnungen und Verknüpfungen zwischen Klassen empfand ich als fehleranfällig. Einerseits bietet diese Struktur viele Möglichkeiten und eine hohe Flexibilität. Andererseits entstehen dadurch auch viele Stellen, an denen kleine Konfigurationsfehler grosse Auswirkungen haben können. Auch die Visualisierung war anspruchsvoller als erwartet. Die Verbindung zwischen SPS-Variablen, Servern, Textschemes und HMI-Elementen musste sehr genau aufgebaut werden. Dadurch wurde die Inbetriebnahme teilweise aufwendig. Gleichzeitig konnte ich dadurch besser nachvollziehen, wie stark SPS-Logik und Visualisierung voneinander abhängig sind und wie wichtig saubere Schnittstellen sind.

Im Gesamten finde ich das Programmierkonzept von SIGMATEK trotzdem interessant und leistungsfähig. Die objektorientierte Struktur bietet viele Vorteile, sobald das Grundprinzip verstanden ist. Für mich wirkte die Umgebung jedoch weniger direkt und komfortabel als andere Systeme, weil sehr viele Einstellungen manuell gepflegt werden müssen.

Trotz dieser Herausforderungen bin ich froh, diese Erfahrung gemacht zu haben. Ich konnte mein Verständnis für Steuerungsarchitektur, objektorientierte SPS-Programmierung, Motion-Anbindung, HMI-Kommunikation und Fehlersuche deutlich erweitern. Wenn ich in Zukunft wieder mit SIGMATEK arbeite, werde ich mit deutlich mehr Verständnis und Sicherheit an die Aufgabe herangehen. 😊

5.2. Hassan

Ich persönlich habe die Case Study als sehr positiv und lehrreich erlebt. Besonders motivierend war für mich zu sehen, wie sich aus einer ersten Idee Schritt für Schritt eine funktionierende Steuerung entwickelt hat. Am Anfang bestand das Projekt vor allem aus Planung, Skizzen und einzelnen Überlegungen. Mit der Zeit wurden daraus konkrete Klassen, Verbindungen, HMI-Elemente und eine Steuerungslogik, die getestet werden konnte.

Für mich war auch die Zusammenarbeit im Team sehr wertvoll. Mein Teammitglied hatte bereits mehr Erfahrung im Entwerfen von Steuerungen aus seinem beruflichen Umfeld. Davon konnte ich stark profitieren, da ich dadurch viele Zusammenhänge besser verstehen konnte. Gleichzeitig konnte ich meine eigenen Aufgaben, besonders im Bereich HMI und Visualisierung, selbstständig bearbeiten und weiterentwickeln.

Spannend war auch, dass wir nicht immer am gleichen Ort gearbeitet haben. Teilweise arbeitete jeder an einem anderen Teil des Projekts, und trotzdem entstand am Ende ein gemeinsames Ergebnis. Durch die Abstimmung über Git, die gemeinsamen Besprechungen und das laufende Testen konnten wir die einzelnen Teile zusammenführen. Insgesamt war die Case Study für mich eine gute Erfahrung, weil ich viel über SPS-Programmierung, HMI-Anbindung, Git und die Zusammenarbeit in einem technischen Projekt gelernt habe.

Aufwandsübersicht

Was	Wer	Zeit
Git-Repo erstellt und getestet	Hassan und Niki	1h
Flussdiagramm erstellt	Hauptsächlich Niki	2h
Dokuvorlage erstellt	Hassan	0.5h
Doku Einleitung, Ziele und Planung geschrieben	Hassan und Niki	3h
Zustandsautomat Vorlage erstellt	Hassan	1h
Zustandsautomat vervollständigt	Niki	3h
Projekt Architektur	Hassan und Niki	1h
Säuberung Analyse	Niki	1h
LASAL-Projekt erstellt für Git-Tests	Hassan und Niki	0.5h
Testklasse für Git-Test	Hassan	0.5h
Steuerung geholt und getestet	Hassan	2h
Architektur, Klasse, in LASAL erstellt	Hassan und Niki	1.5h
Motion und Motorenanbindung	Hassan und Niki	2h
Problematik Bibliothekenübergabe git	Hassan	1h
Git tests und testen Steuerung	Hassan	2h
Methodenübergabe zwischen Klassen erstellt	Niki	1h
IO übergabe von HMI auf Aufzugmodul	Niki	1h
Flankenerkennung DI	Niki	1h
Schrittkettenablauf mit Timer erstellt	Niki	3h
Timerbug wegen Cyclictime	Niki	0.5h
IRL test und debugging library	Niki und Hassan	1h
Endrechner bestimmt und Libraries fertiggestellt	Niki und Hassan	0.5h
VISU getestet	Niki und Hassan	1h
Server für HMI angepasst	Niki und Hassan	0.5h

VISU nach Skizze editiert und Knöpfe mit Server hinterlegt	Hassan	1h
Ablaufkette Motorenansteuerung	Niki	2h
VISU knöpfe gefixed	Hassan	1h
Ist-Etage bei Status angebunden	Hassan	0.5h
Antriebsansteuerung angepasst	Niki	1h
Serverschnittstellen für HMI erweitert	Niki	1h
Türstatus	Hassan	0.5h
Neue statemachine für Motoren	Niki	2h
Textscheme für Richtung	Hassan	0.5h
Troubleshooting HMI rendert Buttons nicht	Hassan	1h
Verlinkung Textscheme	Hassan	1h
Zentrierung Buttons und Outputs	Hassan	1h
clsAntrieb in 2 Objekte unterteilt	Niki	1h
Enum Motorenansteuerung und Bugfix Tür	Niki	2h
Neue Visuanbindung	Niki	1h
Anbindung Richtung an Klasse clsAntrieb	Niki	2h
Neue Server an HMI angebunden und getestet	Hassan	0.5h
Inbetriebnahme an Steuerung	Niki	1h
Bugfix clsHMI	Niki	1h
Notstoppfunktion	Niki	2h
Lichtschranke	Niki	1h
Server für Notstopp und Lichtschranke für Visu	Niki	1h
Bugfix Cycletask	Niki	1h

sZieletage mit Logik und Visualisierung implementiert	Hassan	1h
LED für Zieletage implementiert	Hassan	1h
Debugging Safetycard und Lichtschranke	Hassan	2h
Status LED für Störungen erstellt	Hassan	1h
Logik für die Status Leds verbessert	Hassan	0.5h
Letzte Tests und Einstellungen	Hassan	3h
Korrektur Lesen	Hassan	2h
Präsentation	Hassan	1h
Gesamt Doku	Hassan und Niki	24h

Summe: ca. 90h

Selbständigkeitserklärung

Ich versichere, dass die vorliegende Arbeit selbständig und ohne Benutzung anderer als der angegebenen Hilfsmittel angefertigt wurde. Alle Inhalte dieser Arbeit, dazu gehören neben Texten auch Grafiken, Programmcode, etc., die wörtlich oder sinngemäss aus anderen Quellen stammen, sind als solche eindeutig kenntlich gemacht und korrekt im Quellenverzeichnis gelistet. Dies gilt auch für einzelne Auszüge aus fremden Quellen.

Die Arbeit ist in gleicher oder ähnlicher Form noch nicht veröffentlicht und noch keiner Prüfungsbehörde vorgelegt worden.

Datum: Landquart, 24.06.2026

Unterschrift:

Two handwritten signatures in blue ink. The first signature is on the left, and the second, more stylized signature is on the right.

Abbildungsverzeichnis

Abbildung 1: Flussdiagramm der Aufzugssteuerung	4
Abbildung 2: Zustandsautomat der Aufzugssteuerung	5
Abbildung 3: Architektur der Aufzugssteuerung.....	6
Abbildung 4: HMI-Skizze.....	7
Abbildung 5: Ordnerstruktur des GitHub-Repositorys.....	9
Abbildung 6:Hauptnetzwerk der SPS-Architektur in LASAL	9
Abbildung 7: Eingebundene Motion-Objekte für die Motoranbindung	10
Abbildung 8: Ergebnis HMI	13
Abbildung 9: Git-Network	14

Quellenverzeichnis

- [1] Modul Steuerungstechnik II: Wegleitung zu Case Study Steuerungstechnik, Case Study IV, 2026. Unterrichtsunterlage.
- [2] Modul Steuerungstechnik II: Bewertung Case Study IV Steuerungstechnik II, Bewertungsraster zur Case Study IV, 20.05.2026. Unterrichtsunterlage.
- [3] Modul Steuerungstechnik II: SPS Namenskonventionen – Cheatsheet, 2026. Unterrichtsunterlage.
- [4] Modul Steuerungstechnik II: SCL / ST Cheatsheet – Simatek SPS, 2026. Unterrichtsunterlage.
- [5] Modul Steuerungstechnik II: Datentypen Simatec Steuerungen, 2026. Unterrichtsunterlage.
- [6] Modul Steuerungstechnik II: Programmieren in Structured Text, 31.03.2023. Unterrichtsunterlage.
- [7] Modul Steuerungstechnik II: Anleitung LASAL & VISU Designer, 2026. Unterrichtsunterlage.
- [8] Modul Steuerungstechnik II: Anleitung – Debugger in LASAL / Simatec SCL, 2026. Unterrichtsunterlage.
- [9] Modul Steuerungstechnik II: IBW Simatek Anleitung Motoren, 2026. Unterrichtsunterlage.
- [10] SIGMATEK GmbH & Co KG: LASAL CLASS 2 / VISU Designer Entwicklungsumgebung, verwendete Software für SPS-Programmierung und HMI-Visualisierung, 2026.
- [11] diagrams.net / draw.io: Erstellung der Diagramme und Skizzen für Flussdiagramm, Zustandsautomat, Architekturzeichnung und HMI-Wireframe. <https://www.diagrams.net/>, verwendet im Juni 2026.
- [12] GitHub: Versionsverwaltung des Projektstands und der Dokumentationsdateien. <https://github.com/>, verwendet im Juni 2026.
- [13] OpenAI ChatGPT: Unterstützung bei der sprachlichen Überarbeitung der Dokumentation, bei der Strukturierung der Anforderungen, bei der Ausarbeitung des Planungskapitels sowie bei Vorschlägen für Zustandsautomat, Architekturzeichnung und HMI-Skizze. Verwendet im Juni 2026.
- [14] GitHub Copilot: Unterstützung bei Codevorschlägen, Fehlersuche und technischen Formulierungsvarianten während der Projektarbeit. Verwendet im Juni 2026.

Einsatz von KI-Tools

Während der Bearbeitung wurden KI-Tools unterstützend eingesetzt. OpenAI ChatGPT wurde für die sprachliche Überarbeitung einzelner Dokumentationsabschnitte, die Strukturierung der Anforderungen sowie für Vorschläge zu Planung, Zustandsautomat, Architekturzeichnung und HMI-Skizze verwendet. Die vorgeschlagenen Inhalte wurden überprüft, angepasst und mit der eigenen Aufgabenstellung abgeglichen.

GitHub Copilot wurde unterstützend bei der Programmierung, bei Codevorschlägen und bei der Fehlersuche eingesetzt. Die Vorschläge wurden nicht unverändert übernommen, sondern geprüft und an die verwendete SIGMATEK-SPS, die geplante State Machine und die vorhandene Projektstruktur angepasst.

Die fachlichen Entscheidungen, die Projektidee, die Anforderungen, die Planung, die Umsetzung und die Tests wurden von der Projektgruppe kontrolliert und verantwortet. KI-Tools dienten ausschliesslich als Hilfsmittel zur Unterstützung bei Formulierung, Strukturierung und technischer Ausarbeitung.

Verwendete KI-Prompts

1. Für die Erstellung der HMI-Skizze wurde folgender Prompt verwendet:

„Erstelle eine einfache HMI-Skizze als Wireframe für eine SPS-Aufzugssteuerung mit vier Etagen. Die Skizze soll für ein 1024x600-HMI geeignet sein und schlicht, übersichtlich und technisch wirken. Links soll ein einfacher Lift mit vier Etagen und einer Kabine dargestellt werden. Daneben sollen vier Etagenwahltaster für Etage 1 bis 4 sichtbar sein. Rechts sollen Statusfelder für aktuelle Etage, Ziel-Etage, Fahrtrichtung und Türzustand dargestellt werden. Zusätzlich sollen zwei einfache Taster für „Tür Auf“ und „Tür Zu“ vorhanden sein. Ein kleiner Bereich für Störung/Bonus soll „Notstopp“ und „Lichtschränke“ enthalten. Die Darstellung soll nicht fotorealistisch sein, sondern wie eine technische Konzeptskizze für eine Projektdokumentation aussehen.“

Die erzeugte Skizze wurde geprüft und an die eigene Projektstruktur angepasst.